A) Analysis & evaluation of Object-Oriented design principles & patterns

When I started working on the network management system (NMS), I focused on making the design clean, valid, and flexible. Although the initial source code gave me a solid foundation, I had to extend it to add all the necessary functionality. I made sure that every component of the system was simple to expand and maintain by using object-oriented principles.

One of the first choices I made was to add two new classes: NetworkDevice and DeviceConfiguration. The NetworkDevice class was necessary to show each device in the network with attributes like deviceID and deviceType. By encapsulating these details in a class, I kept the data organised and ensured that other parts of the system could access device details through controlled methods like getdeviceID() and getdevicetype(). This approach also allowed me to handle devices generically, making it easier to extend the system later if new device types need to be added.

The DeviceConfiguration class was added to manage device specific settings, such as IP addresses, mac addresses, or any other configurations. I used a hashmap to store these settings, as it gives flexibility for adding or retrieving key value pairs. Separating configuration logic into its own class made the system modular and adhered to the SRP. For instance, NetworkDevice focuses on representing the device, while DeviceConfiguration handles configuration details. This separation made sure that changes to configuration logic wouldn't affect the rest of the system and vice versa.

To improve logging, I used the gang of four singleton pattern in LoggingManager. This ensures that there is only one instance of the logger across the system, keeping all logs consistent and centralised. Instead of scattered print statements, I used LoggingManager.getInstance() to log messages with appropriate levels like info, warning, and severe. While my current implementation logs to the console using Java's Logger, the design is extensible and can add file logging in the future by including a file handler. This implementation made it easier to track system actions and debug issues, improving the maintainability of the code.

Each class was designed to perform a specific task and interact minimally with others. For example, RouteManager depends on NetworkDeviceManager for retrieving devices but doesn't directly modify the device list. This separation ensured that changes in one class didn't affect others, making the system more robust and easier to test.

I also followed the dependency inversion principle (DIP) to keep the system flexible and separated. For example, RouteManager does not depend on the actual NetworkDevice objects but works with their deviceIDs and the adjacency list. This way, RouteManager doesn't need to know the details of how devices are stored or managed by NetworkDeviceManager. Similarly, NetworkDeviceManager hides its

internal list of devices and provides methods like adddevice() and getdevicebyID() to interact with devices safely. This reliance on abstractions rather than direct dependencies makes it easier to extend the system. For instance, adding a new device type or changing how devices are managed would not affect how RouteManager calculates routes. By using this approach, I ensured that the system is easy to maintain and can adapt to future changes.

I followed the open/closed principle (OCP) to ensure the system was ready for future changes. For example, if new device types like a printer or wireless router need to be added, I can extend the NetworkDevice class without modifying existing code. Likewise, the RouteManager was designed specifically to handle unweighted, bidirectional connections as per task requirements. These choices made the system scalable and adaptable to the given requirements without introducing unnecessary complexity.

In conclusion, I implemented the system this way because I wanted it to be intuitive, reliable, and prepared for future extensions. The use of modularity, logging integration, and design principles like SRP and OCP made the system both functional and maintainable.

B) Explanation of algorithm and time complexity

To calculate the shortest path between two devices, I used the breadth-first search (BFS) algorithm. This was an ideal choice because the network connections were unweighted and bidirectional, and BFS is well-suited for finding the shortest path in such cases.

I started by representing the network as a graph using an adjacency list, which I implemented as a ‘HashMap`. Each key in the map represents a device ID, and its value is a list of connected device IDs. This structure was chosen because it is memory efficient for sparse graphs and allows quick access to neighbours during traversal. Below is the adjacency list, which helped me visualise the network and implement the BFS algorithm.

Table 1: Adjacency list representation of the network

Device	Connected Devices
F1	R1
R1	F1, S1, F2, SW1, SW2
S1	R1
F2	R1, W1
W1	F2, L1, L2
L1	W1

L2	W1
SW1	R1, PC1, PC2, PC3, PC4
PC1	SW1
PC2	SW1
PC3	SW1
PC4	SW1
SW2	R1, PC5, PC6, PC7, PC8
PC5	SW2
PC6	SW2
PC7	SW2
PC8	SW2

The adjacency list table shows how the network is structured and matches the HashMap<String, List<String>> in my RouteManager code. Each device is a key, and its connections are stored as a list of neighbouring devices. I included the table because it makes it easier to visualise the network and understand how the BFS algorithm explores connections layer by layer to find the shortest path. This structure is simple yet efficient for working with the network.

The BFS algorithm works by exploring all connected devices of the source device before moving to their connected devices, layer by layer. I used a queue to keep track of devices to visit and a parentmap to store the path from the source to each device. This map was important for reconstructing the route once the destination was found. If the destination wasn't reachable, the algorithm returned null, and the program displayed a clear message like "No valid route found."

```
while (!queue.isEmpty()) {
    String current = queue.poll();

    if (current.equals(destinationID)) {
        return reconstructPath(parentMap, destinationID);
    }

    for (String neighbor : adjacencyList.get(current)) {
        if (!visited.contains(neighbor)) {
            visited.add(neighbor);
            parentMap.put(neighbor, current);
            queue.add(neighbor);
        }
    }
}
```

Figure 1

The time complexity of BFS is O(V + E), where `V` is the number of devices and `E` is the number of connections. This efficiency made BFS an excellent choice for the

relatively small networks in this coursework. For example, finding the path from `PC8` to `L2` involved exploring only a subset of the graph, which saved time and resources.

The BFS implementation was also flexible enough to accommodate future changes. If weighted connections are introduced, the adjacency list can be modified to store weights, and BFS can be replaced with another algorithm. In summary, BFS provided a simple and effective solution for the current requirements.

C) **Handling different types of inputs**

Ensuring that the system handled a variety of inputs was one of the main focuses during the development process. I designed the program to manage valid, invalid, and edge case inputs while providing clear and informative feedback to users. The testing process was carried out with multiple scenarios, which demonstrated the program's validity and reliability.

For valid inputs, such as finding a route from PC8 to L2 (as shown in figure 4 below), the system successfully parsed the devices.txt and connections.txt files, built the network graph, and executed the BFS algorithm. The output displayed the shortest path as PC8 -> SW2 -> R1 -> F2 -> W1 -> L2. This confirmed that the BFS algorithm functioned correctly by exploring all possible paths efficiently and returning the optimal route.

In cases where invalid inputs were entered, such as missing or incomplete arguments, the program handled the issue well. For example, in Figure 2 below, I tested the system without providing the full argument and missing the end device. Before executing any operations, the program validated the input and immediately identified the missing arguments. It then displayed a severe error indicating that the correct usage of the program requires specifying the file paths for the devices and connections, as well as the starting and ending devices for the route. This check ensured that the program did not proceed unnecessarily, providing helpful feedback to the user while maintaining the stability of the system.

```
(base) sarahalrefaie@Sarahs-Laptop ~ % cd desktop
(base) sarahalrefaie@Sarahs-Laptop desktop % cd src
(base) sarahalrefaie@Sarahs-Laptop src % java -cp . NMS devices.txt connections.txt PC2
[SEVERE] Usage: java NMS <devices.txt> <connections.txt> <startDevice> <endDevice>
(base) sarahalrefaie@Sarahs-Laptop src %
```

Figure 2

In cases where invalid inputs were entered, such as non-existent device IDs, the program handled the issue well. For example, in figure 5 below, I tested a route between PC10 and PC12, devices that do not exist in the network. Before detecting

the invalid device IDs, the program logged the existing devices and routes to confirm that the network was set up correctly. When the invalid device IDs were provided, the program identified them early during validation and skipped the BFS computation entirely. Instead, it displayed a clear error message: “Error: Invalid device IDs provided. No valid route found." This check ensured that the BFS algorithm was not executed unnecessarily.

Another scenario involved disconnected devices where valid IDs existed in the network but had no path between them. For instance, I tested a route between devices located in separate parts of the network that were not connected through any path. In such cases, the program correctly identified that no route existed and displayed a message informing the user. This demonstrated that the BFS algorithm explored all potential connections before concluding that a path could not be found.

I tested the program to ensure it could handle missing or incomplete input files, as shown in Figure 3. The program first successfully processed the devices.txt file, reading each line and adding the devices to the system. However, when the connections.txt file was missing, the program detected this issue and logged a severe error saying the file was not found. Despite this, the program continued to function and attempted to find a route between pc1 and pc3. Since no connections were loaded, it logged warnings of no valid route found and so on. These tests showed that the program could handle missing files, process valid data, and provide clear feedback to users about any issues without crashing.

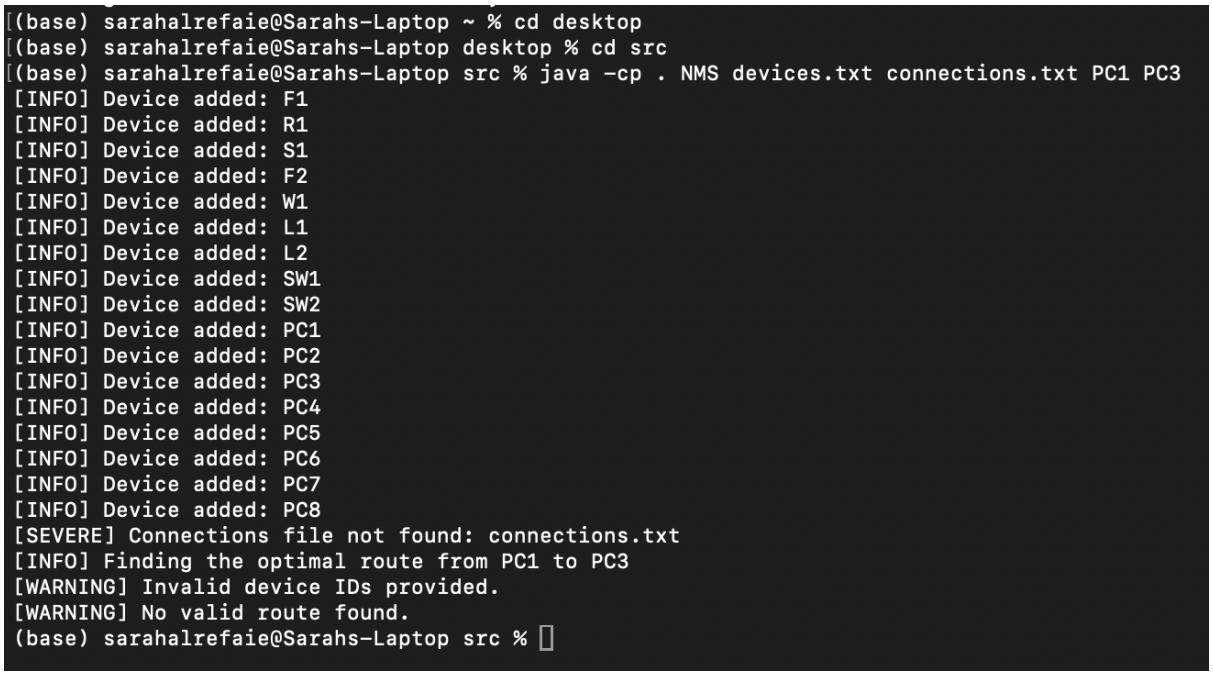

Figure 3

Overall, I tested the program extensively with different inputs: correct device IDs, non-existent devices, and invalid file data. These checks ensured that the system

handled all edge cases well, providing clear messages for invalid inputs while successfully identifying valid routes. By considering these scenarios, I was able to make the program resilient to errors.

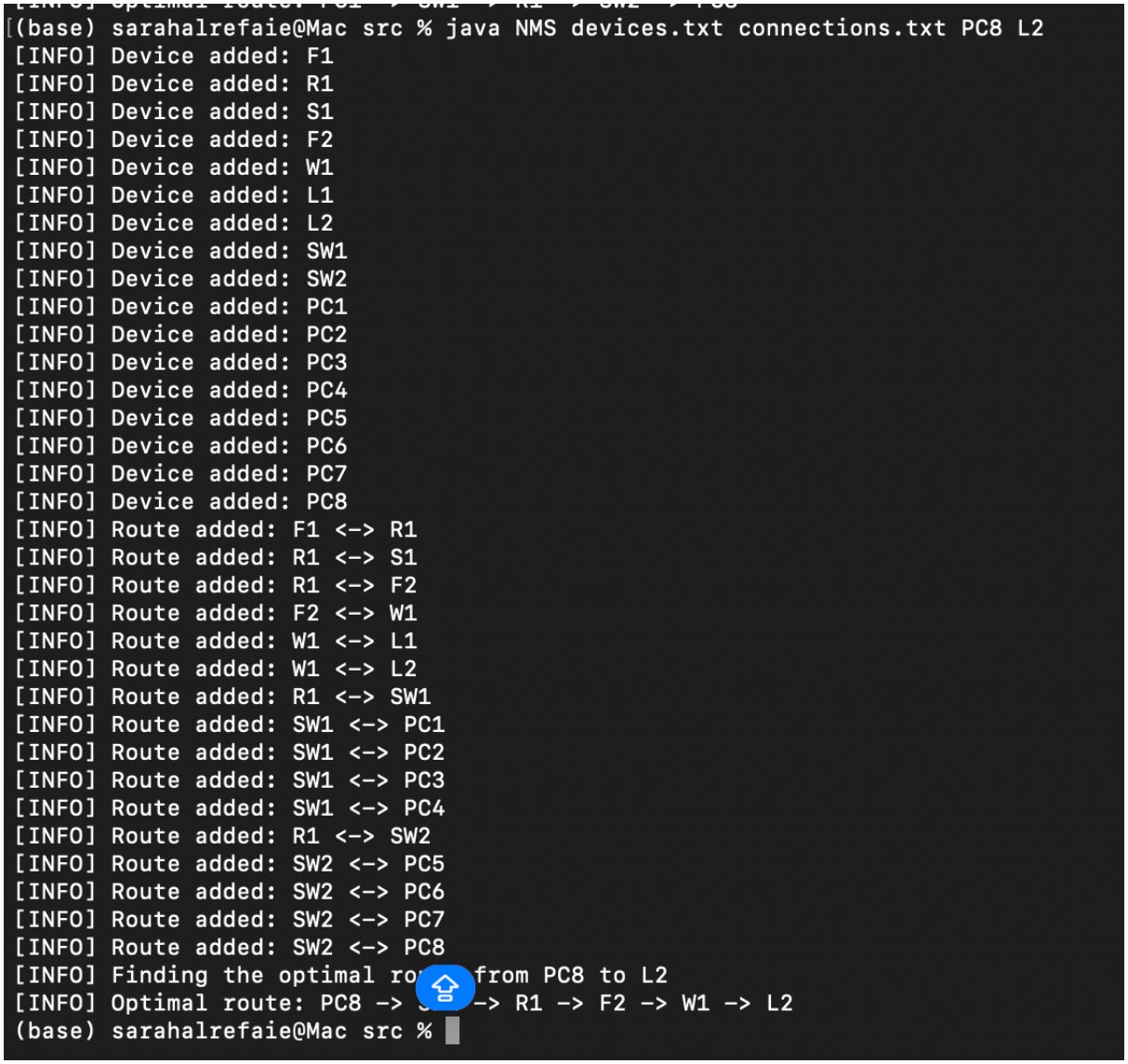

Figure 4

```
(base) sarahalrefaie@Mac src % java NMS devices.txt connections.txt PC10 PC12
[INFO] Device added: F1
[INFO] Device added: R1
[INFO] Device added: S1
[INFO] Device added: F2
[INFO] Device added: W1
[INFO] Device added: L1
[INFO] Device added: L2
[INFO] Device added: SW1
[INFO] Device added: SW2
[INFO] Device added: PC1
[INFO] Device added: PC2
[INFO] Device added: PC3
[INFO] Device added: PC4
[INFO] Device added: PC5
[INFO] Device added: PC6
[INFO] Device added: PC7
[INFO] Device added: PC8
[INFO] Route added: F1 <-> R1
[INFO] Route added: R1 <-> S1
[INFO] Route added: R1 <-> F2
[INFO] Route added: F2 <-> W1
[INFO] Route added: W1 <-> L1
[INFO] Route added: W1 <-> L2
[INFO] Route added: R1 <-> SW1
[INFO] Route added: SW1 <-> PC1
[INFO] Route added: SW1 <-> PC2
[INFO] Route added: SW1 <-> PC3
[INFO] Route added: SW1 <-> PC4
[INFO] Route added: R1 <-> SW2
[INFO] Route added: SW2 <-> PC5
[INFO] Route added: SW2 <-> PC6
[INFO] Route added: SW2 <-> PC7
[INFO] Route added: SW2 <-> PC8
[INFO] Finding the optimal route from PC10 to PC12
[WARNING] Invalid device IDs provided.
[WARNING] No valid route found.
(base) sarahalrefaie@Mac src %
```

Figure 5

D) Impact of future requirements on design

The NMS design was built with future requirements in mind. One of my main goals was to make the system adaptable without requiring major changes, which I believe I achieved through careful planning and flexibility.

If new device types need to be added, the current design can handle this easily. The NetworkDevice class is generic and can be extended to include additional attributes or behaviours. For example, if a printer device is introduced, I can create a subclass of NetworkDevice with the specific properties of a printer. Since NetworkDeviceManager treats all devices generically, it doesn't need any changes to support new types.

Adding weighted connections is another possible requirement. The current adjacency list in RouteManager stores only device IDs, but it can be updated to store weights alongside the IDs. For example, the adjacency list could use a `HashMap<String, Integer>` to store weights. With this change, the BFS algorithm could be replaced with another algorithm like dijktra to calculate the shortest weighted path. This would require updates to RouteManager but wouldn't affect other parts of the system.

Another future requirement could involve calculating paths from one device to all other devices. The BFS implementation is already flexible and can be modified to handle this. Instead of terminating when the destination is found, BFS could continue traversing the entire graph and record distances to all reachable devices.

The system is also designed to handle scalability. The adjacency list representation is efficient for sparse graphs, and the modular design ensures that additional devices or connections can be added without affecting performance significantly. If the network grows too large, optimisations like caching or parallel processing could be introduced to improve efficiency.

In conclusion, the system is well-prepared for future enhancements. By following object-oriented principles and focusing on modularity, I ensured that new features can be added with minimal effort, making the NMS both reliable and future-proof.